# *SIMATS ENGINEERING*

## *ARTIFICIAL INTELLIGENCE – ASSESSMENT TOOL 3*

Assignment Answers

***Course Code: CSA17*** ***Course Outcome: CO2***

***Student Name:*** y.christina

***Register Number:*** 1965260124

***Course Outcome (CO2):***

Demonstrate understanding of uninformed search techniques, including Breadth-First Search, Depth-First Search and state-space search using the 8-puzzle problem.

### *Code of Conduct*

I certify that this submission is my original work and that I have followed the guidelines specified for this assessment.

***Signature of the student:***

y.christina

# *SIMATS ENGINEERING*

ARTIFICIAL INTELLIGENCE (CSA17)
ASSESSMENT TOOL 3 — ANSWERS

Student-friendly answers based on the uploaded CO2_AT3.pdf

## *1. Question 1 – Breadth-First Search (BFS)*

BFS visits nodes level by level. Starting at A, we visit A first, then its children B and C, and then D, E, F, and G. The children are processed from left to right as shown in the graph.

| *Iteration* | *Queue (after processing current node)* | *Visited Node* |
|---|---|---|
| 1 | A | A |
| 2 | B, C | B |
| 3 | C, D, E | C |
| 4 | D, E, F, G | D |
| 5 | E, F, G | E |
| 6 | F, G | F |
| 7 | G | G |

***BFS traversal order:*** A → B → C → D → E → F → G

***Queue after each iteration:*** The queue entries in the table show the remaining nodes after the current node is processed and its unvisited children are added.

## *2. Question 2 – Depth-First Search (DFS)*

DFS follows one branch as far as possible before returning (backtracking) to visit another branch. Taking the graph from left to right, the traversal is A, B, D, E, C, F, G.

| *Iteration* | *Node Visited* | *Stack / action* |
|---|---|---|
| 1 | A | Visit A; go to B |
| 2 | B | Visit B; go to D |
| 3 | D | D has no children; backtrack to B |
| 4 | E | Visit E; backtrack to A |
| 5 | C | Visit C; go to F |
| 6 | F | F has no children; backtrack to C |
| 7 | G | Visit G; finish |

***DFS traversal order:*** A → B → D → E → C → F → G

***Note:*** The uploaded question does not show a specific DFS table layout, so the table above records the node visited and the action at each step.

## *3. Question 3 – 8-Puzzle Using BFS*

***Initial state:***

| | | |
|---|---|---|
| 1 | 3 | 6 |
| 5 | 0 | 2 |
| 4 | 7 | 8 |

***Goal state:***

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 0 |

BFS explores states in order of increasing depth. For the 8-puzzle, each legal move of the blank (0) has equal cost. The first goal state found by BFS is therefore a shortest path. Because the question asks for a trace only through five iterations, the table below shows the first five expansions; the goal is not reached within these five expansions.

| *Iteration* | *Current State* | *Queue Before Expansion* | *Queue After Expansion* |
|---|---|---|---|
| 1 | 136 / 502 / 478 | [S0] | [S1, S2, S3] |
| 2 | 136 / 052 / 478 | [S2, S3] | [S2, S3, S4, S5] |
| 3 | 136 / 520 / 478 | [S3, S4, S5] | [S3, S4, S5, S6, S7] |
| 4 | 106 / 532 / 478 | [S4, S5, S6, S7] | [S4, S5, S6, S7, S8, S9] |
| 5 | 136 / 572 / 408 | [S5, S6, S7, S8, S9] | [S5, S6, S7, S8, S9, S10, S11] |

***Important clarification:*** The provided PDF specifies five table rows but does not define a neighbor-generation order or require the goal to be reached within exactly five expansions. Queue details depend on the order used to generate legal moves. The state notation above uses three rows separated by slashes.

### *3.1 Which node is expanded in each iteration?*

Iteration 1: Initial state S0. Iterations 2–5: the next states are removed from the front of the FIFO queue, in the order shown in the trace table.

### *3.2 How many states are generated in total?*

The total number of states generated cannot be determined from the five-row trace alone. Continue BFS until the goal state is reached, counting each newly discovered unique state and avoiding duplicate states.

### *3.3 Complete solution path from initial state to goal state*

The uploaded sheet does not provide a completed search tree or a required move-order convention. A complete path should be recorded by continuing BFS from the initial state until the goal state is first reached. The five iterations shown are only a partial trace, not a complete solution path.

### *3.4 Why does BFS always find the shortest solution in an unweighted 8-puzzle?*

BFS explores all states at depth 0, then depth 1, then depth 2, and so on. Since every legal move has the same cost, the first time BFS reaches the goal, it has used the fewest moves possible. Therefore, BFS finds a shortest

solution in an unweighted 8-puzzle.